

Word Crush: Türkçe Kelime Tabanlı Mobil Oyun Tasarımı ve Geliştirilmesi

Burak Kılıcı

Bilgisayar Mühendisliği Bölümü

Kocaeli Üniversitesi

Kocaeli, Türkiye

230202111@kocaeli.edu.tr

Özet—Bu çalışmada, Android platformu için Java programlama dili kullanılarak geliştirilen Word Crush adlı kelime tabanlı mobil oyunun tasarımı ve uygulaması sunulmaktadır. Oyun, oyuncuya iki boyutlu grid üzerinde Türkçe harf frekanslarına göre dağıtılmış harflerden anlamlı kelimeler oluşturma olanağı sağlar. Geliştirilen uygulamada Türkçe harf frekans algoritması, komşuluk tabanlı kelime doğrulama, combo mekanizması, özel güç sistemi ve joker mekanizması gibi çeşitli oyun bileşenleri entegre edilmiştir. Veri kalıcılığı için SQLite tabanlı Room Persistence Library kullanılmıştır. Uygulama, kullanıcının hem kelime bilgisini hem de stratejik düşünme yeteneğini geliştirmeyi hedeflemektedir.

Index Terms—mobil oyun, Android, Java, Türkçe kelime oyunu, Room veritabanı, algoritma tasarımı

I. GİRİŞ

Mobil cihazların günlük yaşamın ayrılmaz bir parçası haline gelmesiyle birlikte, mobil uygulamalar ve oyunlar kullanıcıların boş zamanlarını değerlendirmek için en çok tercih ettikleri araçlar arasına girmiştir. Özellikle kelime tabanlı oyunlar, sadece eğlence amacı taşımakla kalmayıp aynı zamanda kullanıcıların dil bilgilerini, kelime haznelerini, problem çözme becerilerini ve bilişsel yetkinliklerini geliştirme konusunda önemli katkılar sağlamaktadır [1]. Wordle, Scrabble, Kelimelik ve benzeri popüler kelime oyunları, milyonlarca kullanıcıya ulaşarak bu alanın ne kadar geniş bir kitleye hitap ettiğini açıkça göstermektedir.

Bu proje kapsamında geliştirilen *Word Crush* adlı mobil oyun, Türkçe kelime hazinesini kullanan, iki boyutlu grid yapısı üzerinden oynanan ve oyuncunun hem kelime bilgisini hem de stratejik planlama yeteneğini ön plana çıkaran bir uygulamadır. Oyun temel olarak, rastgele Türkçe harflerle doldurulmuş bir gridden komşu hücrelerdeki harfler seçilerek anlamlı kelimeler oluşturma prensibi üzerine kuruludur. Oluşturulan geçerli kelimeler grid üzerinden kaldırılır, üstteki harfler yerçekimi etkisiyle aşağı düşer ve boşalan hücrelere yeni harfler üretilir. Oyuncu verilen hamle sayısı içerisinde mümkün olan en yüksek puanı elde etmeye çalışır.

A. Projenin Amacı

Projenin temel amaçları şu şekilde sıralanabilir:

- Mobil programlama konusunda bilgi ve beceri kazanılması
- Android platformu üzerinde Java programlama dili ile oyun geliştirme deneyimi edinilmesi

- Dinamik özelliklere sahip, kullanıcı etkileşimi yoğun bir uygulama geliştirilmesi
- Türkçe dili için özelleştirilmiş algoritmalar tasarlanması
- Veri kalıcılığı, kullanıcı arayüzü ve oyun mantığı konularının entegre şekilde uygulanması

B. Projenin Kapsamı

Geliştirilen uygulama aşağıdaki temel özelliklere sahiptir:

- Kullanıcı adı kayıt ve saklama sistemi
- Üç farklı zorluk seviyesinde grid seçimi (6x6, 8x8, 10x10)
- Türkçe harf frekanslarına uygun akıllı harf üretim algoritması
- Grid üzerinde en az bir geçerli kelime garantisi
- Dokunmatik sürükleme ile 8 yönlü komşuluk tabanlı harf seçimi
- 51.148 kelime içeren Türkçe sözlük ile kelime doğrulaması
- Combo mekanizması ile alt kelime puanlandırması
- Dört farklı uzunlukta kelime için özel güç sistemi
- Altı farklı joker ile oyun içi avantaj sistemi
- SQLite tabanlı veri kalıcılığı ve istatistik takibi
- Geçmiş oyunların detaylı şekilde listelendiği skor tablosu

C. Kullanılan Teknolojiler

Proje aşağıdaki teknolojik altyapı üzerinde geliştirilmiştir:

- **Geliştirme Ortamı:** Android Studio Panda (3.2)
- **Programlama Dili:** Java
- **Minimum SDK:** Android API 24 (Android 7.0)
- **Hedef SDK:** Android API 34 (Android 14)
- **Veritabanı:** Android Room Persistence Library (SQLite tabanlı)
- **UI Bileşenleri:** RecyclerView, ConstraintLayout, LinearLayout
- **Kalıcı Depolama:** SharedPreferences (kullanıcı adı), Room DB (oyun verileri)
- **Test Platformu:** Pixel 4 Emülatörü (Android 11, API 30, x86_64)

D. Raporun Organizasyonu

Raporun geri kalanı şu şekilde organize edilmiştir: Bölüm II sistem tasarımını ve mimari kararları detaylandırır. Bölüm III uygulamada kullanılan algoritmaları ve veri yapılarını anlatır.

Bölüm IV oyun mekaniklerini kapsamlı şekilde sunar. Bölüm V veri yönetimi ve veritabanı tasarımı ele alır. Bölüm VI test sonuçları ve performans analizini içerir. Son olarak Bölüm VII sonuç ve gelecek çalışmalara yönelik önerileri sunar.

II. SİSTEM TASARIMI

Bu bölümde Word Crush uygulamasının genel mimarisi, sınıf yapısı, modül organizasyonu ve kullanıcı arayüzü tasarımı detaylandırılmaktadır. Uygulama, sürdürülebilir ve genişletilebilir bir yapı sağlamak amacıyla katmanlı mimari prensipleri benimsenerek tasarlanmıştır.

A. Uygulama Mimarisi

Uygulama, sorumlulukların net bir şekilde ayrıldığı katmanlı mimari prensibi üzerine kuruludur. Bu yaklaşım, Model-View-Controller (MVC) desenine benzerlik göstermekte olup Android platformunun Activity tabanlı yapısı ile entegre edilmiştir. Katmanlar arası bağımlılık en aza indirgenmiş, her bileşen kendi sorumluluk alanında uzmanlaşmıştır.

Mimarinin temel katmanları şu şekilde özetlenebilir:

- **Sunum Katmanı (View):** XML tabanlı layout dosyaları, görsel kaynaklar ve stiller
- **Kontrolör Katmanı (Activity):** Kullanıcı etkileşimini yöneten ve ekranlar arası geçişi sağlayan Activity sınıfları
- **Mantık Katmanı (Game Logic):** Grid motoru, harf üretici, kelime sözlüğü ve combo hesaplayıcı
- **Veri Katmanı (Data):** Room Persistence kütüphanesi, DAO arayüzleri ve Entity sınıfları
- **Yardımcı Katman (Util):** Çeşitli yardımcı sınıflar ve sabitler

B. Paket ve Sınıf Organizasyonu

Projenin ana paketi `com.worldcrush.game` olarak tanımlanmıştır. Bu ana paket içerisinde işlevsel gruplara göre alt paketler oluşturulmuştur. Bu yapı, kod okunabilirliğini ve sürdürülebilirliğini önemli ölçüde artırmaktadır.

- **activity:** Uygulamanın ekranlarını temsil eden sınıfları barındırır. `SplashActivity`, `MainActivity`, `GridSelectActivity`, `MoveSelectActivity`, `GameActivity`, `ScoreActivity` ve `MarketActivity` sınıflarını içerir.
- **model:** Veri taşıyıcı sınıfları içerir. `Cell` (hücre), `GameRecord` (oyun kaydı), `UserProfile` (kullanıcı profili) ve `Joker` (joker tipi) sınıfları bu paket altındadır.
- **database:** Room Persistence kütüphanesi ile ilgili sınıfları barındırır. `AppDatabase` ve `GameDao` burada yer alır.
- **adapter:** `RecyclerView` yapılarını besleyen adaptör sınıfları. `GridAdapter` ve `GameRecordAdapter` sınıfları bulunur.
- **game:** Oyun mantığının kalbini oluşturan `GridEngine` sınıfını içerir.
- **util:** Yardımcı sınıfları barındırır. `LetterGenerator`, `WordDictionary` ve `ComboHelper` sınıfları bu paket altındadır.

C. Ekran Akışı ve Navigasyon

Uygulama yedi farklı ekrandan oluşmakta olup bu ekranlar arasında mantıksal bir akış tanımlanmıştır. Kullanıcı uygulamayı ilk açtığında **Splash Ekranı** ile karşılaşır. Bu ekranda daha önce kullanıcı adı kaydetmişse doğrudan ana ekrana yönlendirilir, aksi takdirde kullanıcı adı girişi istenir.

Ana Ekran, uygulamanın merkez noktasıdır. Burada üç ana seçenek bulunur: Yeni Oyun, Skor Tablosu ve Market. Kullanıcı adı ekranın sol üst köşesinde görüntülenir ve üzerine tıklanarak değiştirilebilir.

Yeni Oyun seçildiğinde kullanıcı sırasıyla **Grid Boyutu Seçim Ekranı** ve **Hamle Sayısı Seçim Ekranı**'na yönlendirilir. Grid boyutu seçimi oyun zorluğunu doğrudan belirler:

- 10x10 grid: Kolay seviye, 25 hamle hakkı
- 8x8 grid: Orta seviye, 20 hamle hakkı
- 6x6 grid: Zor seviye, 15 hamle hakkı

Hamle sayısı seçimi sonrasında **Oyun Ekranı**'na geçilir. Oyun tamamlandığında veya kullanıcı geri tuşuyla çıktığında sonuçlar veritabanına kaydedilir ve ana ekrana dönlür. Skor Tablosu ve Market ekranları ise ana ekrandan bağımsız olarak erişilebilen bilgilendirme ve satın alma ekranlarıdır.

D. Kullanıcı Arayüzü Tasarım İlkeleri

Kullanıcı arayüzü tasarımında minimalist ve modern bir yaklaşım benimsenmiştir. Arka plan olarak koyu mavi tonları (#1A1A2E ve #16213E) kullanılarak gözü yormayan bir görsel deneyim sağlanmıştır. Vurgu rengi olarak canlı kırmızı (#E94560) tercih edilmiş ve önemli butonlarda kullanılmıştır. Harf kutuları için sıcak sarı tonu (#F4D35E), seçim durumu için ise kırmızı ton kullanılarak görsel geri bildirim güçlendirilmiştir.

Tüm ekranlar arası geçişlerde `overridePendingTransition` kullanılarak yumuşak fade efektleri uygulanmıştır. Harf seçiminde hücreler hafifçe yukarı zıplayıp büyüyerek kullanıcıya anlık görsel geri bildirim sağlanmaktadır.

E. Veri Akışı ve Durum Yönetimi

Uygulamada veri akışı iki ana yönde gerçekleşmektedir: geçici oturum verileri ve kalıcı veriler. Geçici veriler Activity yaşam döngüsü boyunca bellek üzerinde tutulur (örneğin aktif oyundaki puan ve hamle sayısı). Kalıcı veriler ise iki farklı mekanizma ile saklanır:

- **SharedPreferences:** Kullanıcı adı gibi basit anahtar-değer çiftleri için kullanılır. Hızlı erişim sağlar ve uygulama açılışında tek seferde yüklenir.
- **Room Veritabanı:** Oyun kayıtları, kullanıcı profili, altın miktarı ve joker stokları için kullanılır. Karmaşık sorgulara ve ilişkisel veri yapılarına imkan tanır.

F. Sınıflar Arası İlişkiler

Aktivite sınıfları kullanıcı etkileşimlerini yakalar ve bunları `GridEngine` sınıfına aktarır. `GridEngine` tüm oyun mantığını yönetir; harf üretimi için `LetterGenerator`, kelime doğrulama için `WordDictionary` ve kelime arama için kendi içindeki DFS algoritmasını kullanır. Puan hesaplaması

ComboHelper tarafından yapılır. Oyun sonunda elde edilen istatistikler GameDao üzerinden veritabanına işlenir. Bu yapı sayesinde her sınıf kendi sorumluluk alanına odaklanmakta, birbirine gereksiz bağımlılıklar minimize edilmektedir.

III. UYGULAMA VE ALGORİTMALAR

Bu bölümde Word Crush uygulamasının temelini oluşturan algoritmalar ayrıntılı olarak açıklanmaktadır. Geliştirilen algoritmaların performans gereksinimlerini karşılayabilmesi için veri yapıları ve karmaşıklık analizleri dikkatle ele alınmıştır.

A. Türkçe Harf Üretim Algoritması

Bir kelime oyununda harflerin grid üzerine rastgele dağıtılması, oyun deneyimi açısından kritik bir sorun oluşturmaktadır. Tamamen tekdüze rastgele üretim, Türkçe dilinde nadir kullanılan harflerin (J, Ğ, F, V) yoğun şekilde grid üzerinde yer almasına ve kelime oluşturma aşırı zorlaşmasına neden olmaktadır. Bu sorunu çözmek amacıyla Türkçe harf frekanslarına dayalı ağırlıklı rastgele üretim algoritması geliştirilmiştir.

Algoritma 29 Türk alfabesi harfini üç kategoriye ayırır:

- **Sesli harfler:** A, E, İ, I, O, U, Ö, Ü (toplam ağırlık: 45)
- **Yaygın sessizler:** L, R, N, K, M, T, S, Y, D, B (toplam ağırlık: 43)
- **Orta sıklıktaki sessizler:** C, Ç, Ş, Z, H, G, P (toplam ağırlık: 8)
- **Nadir sessizler:** F, V, Ğ, J (toplam ağırlık: 4)

Toplam ağırlık 100 olarak normalize edilmiş olup her harf için kümülatif ağırlık tablosu oluşturulmuştur. Rastgele bir harf üretilmesi istendiğinde 0 ile 100 arasında rastgele bir tamsayı üretilir ve kümülatif tablo üzerinde bu değerin denk geldiği harf seçilir. Algoritmanın karmaşıklığı $O(n)$ olup n harf sayısını temsil etmektedir.

B. Komşuluk Tabanlı Harf Yerleştirme

Yalnızca frekans ağırlıklandırması yeterli değildir; çünkü bir bölgede art arda çok sayıda sessiz harfin toplanması hâlâ mümkündür. Bu problemi çözmek amacıyla grid üretimi sırasında komşuluk kontrolü uygulanmıştır.

Her hücreye harf yerleştirilirken, o hücrenin 8 yöndeki komşularında mevcut olan sesli ve sessiz harf sayıları sayılır. Aday harfin kabul edilmesi için şu kurallar uygulanır:

- Aday sessiz bir harfse ve komşularında 3 veya daha fazla sessiz harf varsa reddedilir.
- Aday sesli bir harfse ve komşularında 3 veya daha fazla sesli harf varsa reddedilir.

Bu kural sayesinde lokal olarak sesli-sessiz dengesi korunur ve kelime oluşturmaya elverişli bir yapı sağlanır. Bir harf en fazla 30 kez denendikten sonra reddedilse bile zorla yerleştirilerek sonsuz döngü engellenir.

C. Kelime Garantisi Algoritması

Grid oluşturulduktan sonra üzerinde hiç kelime bulunmuyorsa oyun başlayamaz. Bu nedenle generateGrid() metodu içerisinde üretilen her grid, hasAtLeastOneWord() metodu ile kontrol edilir. Eğer grid üzerinde en az bir geçerli

Türkçe kelime bulunamıyorsa grid yeniden üretilir. Bu işlem maksimum 20 kez denenir.

Kelime kontrolü için derinlik öncelikli arama (DFS) algoritması kullanılmıştır. Arama derinliği performans kaygısıyla 5 ile sınırlandırılmıştır; çünkü daha derin aramalar $O(8^n)$ karmaşıklık nedeniyle uygulamayı dondurabilmektedir.

D. Kelime Doğrulama Mekanizması

Kelime doğrulaması için 51.148 Türkçe kelime içeren bir sözlük dosyası (kelimeler.txt) uygulamaya assets klasörü altına eklenmiştir. Uygulama ilk açıldığında WordDictionary sınıfı tarafından bu dosya okunur ve tüm kelimeler büyük harfe çevrilerek bir HashSet<String> yapısına yüklenir. Bu sayede:

- Ortalama $O(1)$ arama süresi elde edilir.
- Kelime doğrulama işlemi anlık olarak gerçekleşir.
- Bellek kullanımı kabul edilebilir seviyededir (yaklaşık 3-4 MB).

Türkçe dilinin büyük harf dönüşümü İngilizceden farklı olduğundan (i' harfi için), Locale("tr", "TR") kullanılarak doğru dönüşüm sağlanmıştır. Aksi takdirde "istanbul" kelimesi İSTANBUL' yerine İSTANBUL' olarak hatalı dönüşürdü.

E. Komşuluk Tabanlı Kelime Oluşturma

Oyuncu, parmağını bir harfe dokunarak kelime oluşturma işlemine başlar. Sonrasında parmağını sürükleyerek komşu harfleri seçer. Komşuluk 8 yönde tanımlanmıştır: yukarı, aşağı, sağ, sol, sağ üst, sağ alt, sol üst ve sol alt. İki hücrenin komşu olması için satır ve sütun farklarının mutlak değerinin en fazla 1 olması, ancak aynı hücre olmaması gerekmektedir. Matematiksel olarak:

$$\text{komşu}(a, b) = |a.r - b.r| \leq 1 \wedge |a.c - b.c| \leq 1 \wedge (a \neq b) \quad (1)$$

Touch event sistemi Android'in MotionEvent API'si üzerine kurulmuştur. ACTION_DOWN ile seçim başlar, ACTION_MOVE ile komşu harflere devam edilir, ACTION_UP ile kelime tamamlanır.

F. Kelime Tarama Algoritması (DFS)

Grid üzerinde oluşturulabilecek kelimelerin sayısı her hamle sonrasında ekranda gösterilir. Bu değer countPossibleWords() metodu ile hesaplanır. Algoritma her hücreden başlayarak derinlik öncelikli arama gerçekleştirir.

Temel DFS adımları şu şekildedir:

- 1) Başlangıç hücreni ziyaret edilmiş olarak işaretle.
- 2) Mevcut harfleri bir StringBuilder'a ekle.
- 3) Eğer uzunluk 3 veya daha fazlaysa ve kelime sözlükte varsa bulunan listeye ekle.
- 4) 8 komşu yöne doğru rekürsif çağrı yap.
- 5) Geri dönüşte ziyaret işaretini kaldır (backtracking).

Performans optimizasyonu için derinlik 5 ile sınırlandırılmış ve bulunan kelimeler HashSet ile tekrar sayımı engellenmiştir. Ayrıca bu işlem UI iş parçacığı yerine arka plan iş parçacığında çalıştırılarak ana ekran donmaları önlenmiştir.

G. Yerçekimi ve Harf Yenileme Algoritması

Geçerli bir kelime oluşturulduğunda seçilen harfler grid üzerinden kaldırılır. Bu kaldırma işlemi sonrası gridin hâlâ tam (boşluksuz) kalması için yerçekimi uygulanır. Her sütun için aşağıdaki adımlar gerçekleştirilir:

- 1) Sütundaki mevcut (boş olmayan) harfler alttan yukarıya doğru sırayla okunur.
- 2) Bu harfler sütunun en altından başlanarak yeniden yerleştirilir.
- 3) Üstte kalan boş hücrelere LetterGenerator ile yeni harfler üretilir.

Bu algoritma $O(n^2)$ karmaşıklığa sahiptir; ancak grid boyutu küçük olduğundan (en fazla 100 hücre) işlem anında tamamlanır.

IV. OYUN MEKANİKLERİ

Bu bölümde Word Crush oyununun kullanıcı deneyimini zenginleştiren oyun mekanikleri detaylı şekilde açıklanmaktadır. Her bir mekanik, hem algoritmik hem de kullanıcı arayüzü açısından incelenmektedir.

A. Harf Puanlama Sistemi

Oyunda her Türkçe harfin belirli bir puan değeri bulunmaktadır. Puanlar, harflerin Türkçe dildeki kullanım sıklığı ile ters orantılı olarak belirlenmiştir. Yani nadir kullanılan harfler daha yüksek puan değerine sahiptir. Proje gereksinimlerinde belirtilen harf puanları Tablo I üzerinde gösterilmektedir.

Tablo I
TÜRKÇE HARF PUAN TABLOSU

Harf	Puan	Harf	Puan	Harf	Puan
A	1	K	1	S	2
B	3	L	1	Ş	4
C	4	M	2	T	1
Ç	4	N	1	U	2
D	3	O	2	Ü	3
E	1	Ö	7	V	7
F	7	P	5	Y	3
G	5	R	1	Z	4
Ğ	8	İ	2	J	10
H	5	İ	1		

Bir kelimenin toplam puanı, o kelimeyi oluşturan harflerin puanlarının aritmetik toplamı ile bulunur. Örneğin *ŞÖRU* kelimesinin puanı $S(2) + Ö(7) + R(1) + U(2) = 12$ olarak hesaplanır.

B. Combo Mekanizması

Combo mekanizması, oyuncuyu daha uzun ve karmaşık kelimeler oluşturmaya teşvik eden önemli bir puanlandırma özelliğidir. Oyuncunun oluşturduğu ana kelimenin içinde yer alan tüm geçerli alt kelimeler tespit edilerek her birinin puanı toplama eklenir. Combo mekanizmasının kuralları şu şekildedir:

- 1) Oyuncu grid üzerinde bir ana kelime oluşturur.
- 2) Ana kelimenin içinden 3 harf veya daha uzun tüm ardışık alt diziler taranır.

- 3) Her bir alt dizi sözlükte doğrulanır.
- 4) Doğrulan alt kelimeler combo sayısını artırır.
- 5) Ana kelime her zaman combo'ya dahildir.
- 6) Aynı alt kelime birden fazla kez sayılmaz.

Örnek olarak *ADANA* kelimesi bulunduğunda içindeki *ADANA*, *ADAN*, *ADA*, *DANA*, *DAN*, *ANA* alt dizileri taranır. Bunlardan sözlükte bulunan *ADANA*, *ADA*, *DANA*, *ANA* kelimeleri combo'ya dahil edilir ve toplam puan bu dört kelimenin puanlarının toplamı olarak hesaplanır. Bu mekanizma sayesinde uzun kelime bulan oyuncular ciddi miktarda bonus puan kazanır.

C. Harf Patlatma ve Yerçekimi Mekanikliği

Geçerli bir kelime oluşturulduğunda patlatma mekanikliği devreye girer. Bu mekanik görsel olarak yerçekimi prensibine benzer şekilde çalışır ve oyunun akışkanlığını sağlar. Patlatma süreci şu adımlardan oluşur:

- 1) Seçilen hücrelerin harfleri boşaltılır (özel bir boşluk karakteri atanır).
- 2) Her sütun ayrı ayrı işlenir.
- 3) Sütundaki boş olmayan harfler alttan yukarıya sırayla okunur ve sütunun en altından başlanarak yeniden yerleştirilir.
- 4) Üstte kalan boş hücrelere LetterGenerator aracılığıyla Türkçe harf frekanslarına uygun yeni harfler üretilir.

Bu yaklaşım, oyuncuya harflerin yukarıdan aşağı düştüğü izlenimini vermekte ve klasik match-3 tarzı oyunların akıcılığını sağlamaktadır.

D. Özel Güç Sistemi

Uzun kelime oluşturma ödüllendirilmesi amacıyla özel güç sistemi geliştirilmiştir. 4 harf veya daha uzun kelimeler oluşturulduğunda son harfin yerine özel bir simge bırakılır. Bu simge sonraki bir kelime içinde kullanıldığında ilgili özel güç aktive olur. Tablo II özel güç tiplerini ve etkilerini göstermektedir.

Tablo II
ÖZEL GÜÇ TIPLERİ VE ETKİLERİ

Kelime Uzunluğu	Simge	Etki
4 harf		Harfin bulunduğu satır tamamen temizlenir
5 harf		Harfin çevresindeki 3x3 alan patlatılır
6 harf		Harfin bulunduğu sütun tamamen temizlenir
7+ harf		5x5 alan mega patlatma ile yok edilir

Özel güçler zincir tepkimesi yaratabilir. Örneğin bir satır temizleme sırasında içinde sütun temizleme simgesi olan bir hücre bulunuyorsa, o da aktive olmalıdır. Ancak performans ve karmaşıklık dengesi gözetilerek şu anki uygulamada zincir tetiklemesi yerine tek seviyeli aktivasyon tercih edilmiştir.

E. Joker Sistemi

Oyun deneyimini zenginleştirmek için altı farklı joker sistemi tasarlanmıştır. Jokerler Market ekranından oyun içi altın kullanılarak satın alınır ve oyun sırasında ekranın altındaki butonlarla aktive edilir. Her jokerin farklı bir oyun stratejisi katkısı vardır:

- **Bahk (100 altın):** Grid üzerindeki rastgele seçilen 5 harfi yok eder. Yok edilen harflerin üzerindeki harfler aşağıya düşer ve boş kalan hücrelere yeni harfler üretilir. Grid'in dinamiğini hızlıca değiştirmek için kullanışlıdır.
- **Tekerlek (200 altın):** Kullanıcının seçtiği bir harfin bulunduğu satır ve sütundaki tüm harfleri yok eder. En güçlü yok etme jokerlerinden biridir.
- **Lolipop Kırıcı (75 altın):** Tek bir harfi hassas şekilde yok etmek için kullanılır. En ucuz joker olup stratejik planlamada oldukça etkilidir.
- **Serbest Değiştirme (125 altın):** Kullanıcının seçtiği iki komşu harfin yerini değiştirir. Kelime oluşturmak için yakın ama uyumsuz harfleri düzenlemeye yarar.
- **Harf Karıştırma (300 altın):** Grid üzerindeki tüm mevcut harflerin yerini rastgele karıştırır. Yeni harf üretilmez, sadece mevcut harfler yeniden dağıtılır.
- **Parti Güçlendirici (400 altın):** Grid üzerindeki tüm harfleri yok eder ve tamamen yeni harflerle doldurur. En pahalı ama en kapsamlı jokerdir.

Proje gereksinimleri doğrultusunda gerçek para ile altın satın alma sistemi uygulamaya dahil edilmemiştir. Bunun yerine kullanıcıya başlangıçta 99999 altın tanımlanmış ve test sırasında altın 5000'in altına düştüğünde otomatik olarak yenilenen bir mekanizma kurulmuştur. Bu yaklaşım, kullanıcı deneyiminin kesintisiz sürmesini sağlamaktadır.

F. Grid Oynanabilirlik Kontrolü

Her hamle sonrasında grid üzerinde hâlâ kelime oluşturulup oluşturulamayacağı kontrol edilir. Bu kontrol arka planda çalışan bir iş parçacığı üzerinde yapılarak kullanıcı arayüzünün donmasının önüne geçilmiştir. Eğer sistem grid üzerinde hiç kelime tespit edemezse iki aşamalı bir çözüm mekanizması devreye girer:

- 1) **Karıştırma (Shuffle):** Gridde mevcut olan harfler rastgele yeniden dağıtılır. Bu işlem maksimum 10 kez denir.
- 2) **Tam Yenileme:** Eğer karıştırma başarısız olursa grid baştan üretilir. Bu durum istisnaidir çünkü grid zaten başlangıçta kelime garantisi ile üretilmektedir.

G. Oyun Bitişi ve Skor Kaydı

Oyun iki şekilde sona erebilir:

- Hamle sayısının sıfıra inmesi
- Kullanıcının geri tuşuna basarak oyundan çıkmayı seçmesi

Her iki durumda da oyun sonuçları otomatik olarak veritabanına kaydedilir. Hamleler bittiğinde oyuncuya özet içeren bir dialog gösterilir. Bu dialog toplam puanı, bulunan kelime sayısını ve en uzun kelimeyi içerir. Geri

tuşuna basıldığında ise onay dialog'u çıkar ve kullanıcıya çıkmak istediğinden emin olup olmadığı sorulur.

V. VERİ YÖNETİMİ

Veri yönetimi, uygulamanın kalıcı durumunu sağlayan kritik bir bileşendir. Word Crush uygulamasında iki farklı veri depolama mekanizması kullanılmaktadır: *SharedPreferences* basit anahtar-değer çiftleri için, *Room Persistence Library* ise yapısal veri depolama için tercih edilmiştir.

A. SharedPreferences Kullanımı

SharedPreferences, Android platformunun sağladığı hafif bir anahtar-değer depolama sistemidir. XML tabanlı çalışır ve küçük boyutlu basit veriler için idealdir. Uygulama içinde yalnızca kullanıcı adı bilgisini saklamak için kullanılmaktadır. Kullanıcı adının bu yolla saklanması iki temel nedeni vardır:

- Uygulama açılışında tek bir okuma ile hızlıca erişilebilmesi
- *Splash Activity*'de kullanıcı adının varlığının senkron olarak kontrol edilebilmesi

B. Room Persistence Library

Room, Google tarafından geliştirilen ve *SQLite* üzerine soyutlama katmanı sağlayan bir kütüphanedir. Derleme zamanında SQL sorgularını kontrol eder, veri sınıflarını (Entity) otomatik olarak tablolara dönüştürür ve tip güvenli bir API sunar. Word Crush uygulamasında *Room* kullanılması temel nedenleri şunlardır:

- Tip güvenliği ile derleme zamanında hata tespiti
- DAO desenine uygun temiz bir mimari
- Sorgu cache'leme ve iş parçacığı yönetimi
- Veritabanı şema migration desteği

C. Veritabanı Şeması

Uygulamada iki ana tablo tanımlanmıştır:

user_profile tablosu, kullanıcı ile ilgili tüm kalıcı bilgileri saklar. Alanları şunlardır:

- **id:** Birincil anahtar, otomatik artan
- **username:** Kullanıcı adı (String)
- **totalGold:** Kullanıcının sahip olduğu altın miktarı (int)
- **totalGames, highScore, totalWords:** Genel istatistikler
- **jokerFish, jokerWheel, jokerLollipop, jokerSwap, jokerShuffle, jokerParty:** Her joker türü için stok sayıları

game_records tablosu, oynanan her oyunun detaylarını saklar. Alanları şunlardır:

- **id:** Birincil anahtar, otomatik artan
- **username:** Oyunu oynayan kullanıcının adı
- **score:** Oyun sonu puanı
- **wordCount:** Bulunan kelime sayısı
- **longestWord:** En uzun bulunan kelime

- **gridSize:** Kullanılan grid boyutu (6, 8 veya 10)
- **moveCount:** Kalan hamle sayısı
- **duration:** Oyun süresi (saniye cinsinden)
- **date:** Oyun tarihi (Unix timestamp)

D. DAO Yapısı

Data Access Object (DAO) deseni ile veritabanı işlemleri tek bir arayüz altında toplanmıştır. GameDao arayüzü tüm CRUD (Create, Read, Update, Delete) operasyonlarını içerir. Öne çıkan sorgular:

- **@Insert:** Yeni oyun kaydı ve kullanıcı ekleme
- **@Query("SELECT * FROM game_records ORDER BY date DESC"):** En yeni oyunlar üstte olacak şekilde listeleme
- **@Query("UPDATE user_profile SET totalGold = :gold ..."):** Altın miktarını güncelleme
- Her joker tipi için ayrı artırma ve kullanma sorguları

E. Veri Tutarlılığı ve İş Parçacığı Yönetimi

Room varsayılan olarak ana iş parçacığında veritabanı erişimine izin vermez. Ancak basit uygulamalarda bu kısıtlama `allowMainThreadQueries()` metodu ile kaldırılabilir. Bu projede performans açısından kritik olmayan okuma işlemleri için bu yaklaşım tercih edilmekle birlikte, yazma işlemleri ve karmaşık sorgular için `new Thread()` yapısı kullanılarak arka plan iş parçacıkları oluşturulmuştur. `runOnUiThreadThread()` metodu ile sonuçlar ana iş parçacığına aktarılmıştır.

F. Şema Evrimi (Migration)

Uygulama geliştirme sürecinde veritabanı şeması bir kez güncellenmiştir. Başlangıçta sadece temel kullanıcı bilgileri bulunurken, joker sistemi eklendiğinde 6 adet yeni alan (`jokerFish`, `jokerWheel` vb.) eklenmiştir. Bu değişiklik için veritabanı versiyonu 1'den 2'ye yükseltilmiş ve `fallbackToDestructiveMigration()` yaklaşımı benimsenmiştir. Bu yaklaşımda şema uyumsuzluğu durumunda eski veriler silinerek yeni şema oluşturulmaktadır. Gerçek ürün ortamında daha dikkatli bir migration stratejisi tercih edilmelidir; ancak geliştirme ortamı için bu yaklaşım pratiktir.

VI. TEST VE PERFORMANS ANALİZİ

Bu bölümde geliştirilen Word Crush uygulamasının test süreci, performans ölçümleri ve elde edilen sonuçlar detaylı şekilde sunulmaktadır. Testler, farklı grid boyutları ve oyun senaryoları üzerinde gerçekleştirilmiştir.

A. Test Ortamı

Uygulama, Android Studio'nun sağladığı Android Virtual Device (AVD) aracılığıyla oluşturulan emülatör üzerinde test edilmiştir. Test ortamına ait teknik özellikler şunlardır:

- **Emülatör Cihazı:** Google Pixel 4
- **İşletim Sistemi:** Android 11.0 (API Level 30)

- **Mimari:** x86_64 (Google Play desteğiyle)
- **RAM:** 1536 MB
- **Çözünürlük:** 1080 x 2280 piksel, 440 dpi
- **Multi-Core CPU:** 4 çekirdek

Emülatör, Intel HAXM donanım hızlandırması ile çalıştırılmış ve sanallaştırma desteği aktif tutulmuştur. Bu sayede uygulama, gerçek bir cihaza yakın performans göstermiştir.

B. Grid Üretim Performansı

Grid üretim algoritmasının performansı üç farklı grid boyutu için ölçülmüştür. Ölçümler 100 tekrarın ortalaması alınarak hesaplanmıştır. Sonuçlar Tablo III üzerinde gösterilmektedir.

Tablo III
GRID ÜRETİM PERFORMANS ÖLÇÜMLERİ

Grid Boyutu	Hücre	Ortalama Süre	Olası Kelime
6x6	36	~15 ms	150-300
8x8	64	~25 ms	250-500
10x10	100	~45 ms	400-800

Görüldüğü üzere 10x10 grid için bile üretim süresi 50 ms altında kalmaktadır. Bu, kullanıcı deneyimini olumsuz etkileyecek bir gecikmeye neden olmamaktadır. Olası kelime sayısının grid boyutu büyüdükçe artması beklenen bir sonuçtur; çünkü harf sayısı arttıkça kombinasyon olasılığı üstel olarak yükselir.

C. Kelime Doğrulama Performansı

Kelime doğrulama işlemi `HashSet` veri yapısı kullanılarak gerçekleştirildiği için ortalama karmaşıklık $O(1)$ 'dir. Yapılan 10.000 rastgele kelime doğrulama testinde toplam süre 35 ms olarak ölçülmüştür. Başına kelime başına doğrulama süresi yaklaşık **3.5 mikrosaniye** olarak hesaplanmıştır. Bu değer, oyun akışını etkilemeyecek kadar küçüktür.

D. Kelime Tarama (DFS) Performansı

Grid üzerindeki olası kelime sayısını hesaplayan DFS algoritması, derinlik sınırının 5 olarak belirlenmesiyle kabul edilebilir sürelerde çalışmaktadır. Performans karşılaştırması Tablo IV üzerinde verilmiştir.

Tablo IV
FARKLI DERİNLİKLERDE DFS PERFORMANSI (10X10 GRID)

Derinlik	Tarama Süresi	Bulunan Kelime
3	12 ms	~150
5	180 ms	~500
6	950 ms	~700
7	5200 ms	~850
8	28000 ms	~950

Görüldüğü gibi derinlik arttıkça süre üstel olarak büyümektedir. Bu nedenle derinlik 5 ile sınırlandırılmış, kullanıcı deneyimi ve bulunan kelime sayısı arasında optimum denge sağlanmıştır. Ayrıca bu işlem ana iş parçacığı yerine arka plan iş parçacığında çalıştırılarak UI dondurulmaları engellenmiştir.

E. Harf Dağılım Analizi

Türkçe harf frekans algoritmasının etkinliğini test etmek amacıyla 1000 adet 10x10 grid üretilerek harf dağılımları analiz edilmiştir. Elde edilen sonuçlar, hedeflenen frekansa oldukça yakındır. Örneğin sesli harfler toplamda %45 oranında üretilmiştir ki bu Türkçe dilinin doğal yapısına uygundur. Nadir harflerin (J, Ğ) yoğunluğu her grid için ortalama 1-2 adet civarında kalmıştır.

F. Bellek Kullanımı

Uygulamanın bellek kullanımı Android Studio Profiler aracı ile incelenmiştir. Temel gözlemler şu şekildedir:

- **Sözlük yükleme:** 51.148 kelime HashSet'e yüklenirken yaklaşık 4 MB bellek tüketmektedir.
- **Grid durumu:** 10x10 grid için ayrılan bellek miktarı 1 KB'nin altındadır.
- **Aktif Activity:** Oyun ekranı çalışırken toplam bellek kullanımı yaklaşık 55-60 MB civarındadır.
- **Veritabanı:** Room veritabanı dosya boyutu 100 oyun kaydı için 50 KB altındadır.

G. Kullanıcı Deneyimi Testleri

Uygulama, gerçek kullanıcı senaryoları üzerinden manuel olarak test edilmiştir. Test edilen temel senaryolar şunlardır:

- a) **İlk Açılış Senaryosu:** Kullanıcı adı alma, veritabanı oluşturma, ana ekrana yönlendirme.
- b) **Yeni Oyun Başlatma:** Grid boyutu ve hamle seçimi, oyun ekranına geçiş, grid oluşumu.
- c) **Harf Seçimi ve Kelime Oluşturma:** Dokunma ve sürükleme ile harf seçimi, komşuluk kontrolü, kelime doğrulama.
- d) **Özel Güç Aktivasyonu:** 4-7+ harfli kelimelerle özel güç simgelerinin oluşturulması ve kullanılması.
- e) **Joker Kullanımı:** Market'ten joker satın alma, oyun içinde kullanma, stok güncelleme.
- f) **Oyun Bitişi:** Hamle bitimi sonrası skor kaydı, skor tablosunda görüntüleme.

Tüm senaryolar başarıyla tamamlanmıştır. Beklenmedik durum testlerinde (arka plana alma, ekran döndürme, beklenmedik kapanış) uygulama kararlılığını korumuştur.

H. Karşılaşılan Zorluklar ve Çözümler

Geliştirme süreci boyunca karşılaşılan başlıca zorluklar ve bunlara üretilen çözümler aşağıda özetlenmiştir:

- **Sorun:** Tamamen rastgele harf üretimi ile kelime oluşturma imkansızlaşması.
Çözüm: Türkçe harf frekans ağırlıklandırması ve komşuluk tabanlı sesli-sessiz dengesi.
- **Sorun:** DFS kelime tarama algoritmasının ana iş parçacığını dondurmaları.
Çözüm: Derinlik sınırı (5) ve arka plan iş parçacığına taşıma.

- **Sorun:** Emülatörde ARM mimarisinin desteklenmesi.
Çözüm: x86_64 tabanlı sistem imajı kullanılması.
- **Sorun:** Türkçe büyük-küçük harf dönüşümünde 'ı' harfinin yanlış dönüşmesi.
Çözüm: `Locale("tr", "TR")` kullanılarak doğru dil ayarı ile dönüşüm.
- **Sorun:** Veritabanı şema değişikliğinde uygulamanın çökmesi.
Çözüm: Versiyon yükseltme ve `fallbackToDestructiveMigration()` stratejisi.
- **Sorun:** Grid üzerinde hiç kelime kalmaması olasılığı.
Çözüm: Otomatik karıştırma ve gerekirse yeniden üretme mekanizması.

VII. SONUÇ VE GELECEK ÇALIŞMALAR

Bu çalışmada Android platformu için Java programlama dili kullanılarak Word Crush adlı Türkçe kelime tabanlı mobil oyun başarıyla geliştirilmiştir. Proje kapsamında belirlenen tüm gereksinimler karşılanmış, oyun akışkan ve hatasız biçimde çalışır hale getirilmiştir.

A. Elde Edilen Başarılar

Gerçekleştirilen çalışma ile aşağıdaki başarılar elde edilmiştir:

- Türkçe harf frekansına uygun ve dengeli harf üreten akıllı bir grid algoritması tasarlanmıştır.
- 51.148 kelimeden oluşan bir sözlük ile anlık kelime doğrulaması sağlanmıştır.
- 8 yönlü komşuluk tabanlı dokunmatik harf seçim sistemi başarıyla uygulanmıştır.
- Combo mekanizması ile alt kelimelerin tespiti ve puanlandırması çalışmaktadır.
- Dört farklı seviyede özel güç ve altı farklı joker sistemi entegre edilmiştir.
- Room Persistence Library ile profesyonel düzeyde veri kalıcılığı sağlanmıştır.
- Modern ve kullanıcı dostu bir arayüz tasarımı gerçekleştirilmiştir.

B. Projenin Sağladığı Kazanımlar

Proje süreci boyunca mobil uygulama geliştirme alanında önemli kazanımlar elde edilmiştir. Android Studio ortamının kullanımı, Activity yaşam döngüsü yönetimi, RecyclerView ile performanslı liste görüntüleme, Room ORM kullanımı, XML tabanlı arayüz tasarımı ve farklı Android versiyonları için uyumluluk sağlama gibi konularda derinlemesine deneyim kazanılmıştır. Ayrıca algoritma tasarımı, veri yapısı seçimi ve performans optimizasyonu konularında teorik bilginin pratik uygulamaya aktarılması sağlanmıştır.

C. Projenin Sınırları

Bu çalışmanın bazı sınırları bulunmaktadır. Kelime tarama algoritmasında derinlik sınırı 5 olarak belirlendiği için daha uzun kelimeler tespit edilememektedir. Animasyon sistemi minimum seviyede tutulmuş olup daha zengin görsel efektler eklenebilir. Ayrıca uygulama yalnızca Türkçe dilini desteklemekte olup başka dillere genişletilmesi için çalışma gerekmektedir.

D. Gelecek Çalışmalar

Projenin daha da geliştirilmesi amacıyla aşağıdaki çalışmalar önerilebilir:

- **Çoklu Dil Desteği:** İngilizce, Almanca, Fransızca gibi farklı dillere genişletme. Her dil için frekans tabloları ve sözlük dosyaları eklenerek dinamik dil değişikliği sağlanabilir.
- **Çok Oyunculu Mod:** Firebase veya benzeri backend servislerle gerçek zamanlı iki oyunculu mod geliştirilebilir. Oyuncular aynı grid üzerinde yarışabilir.
- **Gelişmiş Animasyonlar:** Harflerin düşme animasyonu, patlama efektleri, parçacık sistemi ve ses efektleri eklenebilir.
- **Başarım Sistemi:** Belirli hedefleri tamamlayan oyuncular için rozet ve başarımlar sistemi uygulanabilir.
- **Günlük Görevler:** Her gün farklı hedefler sunan görev sistemi ile oyuncu bağlılığı artırılabilir.
- **Zorluk Ayarları:** Grid boyutuna ek olarak kelime sayısı, kelime uzunluğu minimum/maksimum ayarları gibi gelişmiş zorluk seçenekleri sunulabilir.
- **Günlük Mücadele:** Tüm oyuncuların aynı grid üzerinde oynayacağı günlük mücadele modu ile rekabet artırılabilir.
- **Yapay Zeka Destekli İpuçları:** Oyuncunun zorlandığı durumlarda olası kelimeleri gösteren bir ipucu sistemi eklenebilir.
- **iOS Versiyonu:** Uygulamanın iOS platformuna taşınması ile hedef kitle genişletilebilir. Bu amaçla Kotlin Multiplatform veya Flutter gibi çapraz platform teknolojileri değerlendirilebilir.
- **Bulut Senkronizasyonu:** Kullanıcıların oyun verilerini bulutta saklayarak farklı cihazlardan erişimi sağlanabilir.
- **İstatistik Analizi:** Kullanıcının en çok bulduğu harfler, ortalama kelime uzunluğu, başarı trendleri gibi detaylı istatistikler sunulabilir.

E. Genel Değerlendirme

Word Crush projesi, mobil programlama eğitiminde verilen teorik bilginin uygulamaya dönüştürülmesi açısından başarılı bir örnek oluşturmaktadır. Proje kapsamında kullanılan algoritma tasarımı, veri yapısı seçimi, kullanıcı arayüzü tasarımı ve veritabanı yönetimi gibi unsurlar, profesyonel düzeyde mobil uygulama geliştirme süreçlerinin temel bileşenlerini oluşturmaktadır. Geliştirilen

uygulama, hem eğitim hem de eğlence amacıyla kullanılabilir bir ürün olarak tamamlanmıştır ve gelecek çalışmalarla daha da zenginleştirilebilir potansiyele sahiptir.

KAYNAKLAR

- [1] R. M. Ryan and E. L. Deci, "Intrinsic and extrinsic motivations: Classic definitions and new directions," *Contemporary Educational Psychology*, vol. 25, no. 1, pp. 54-67, 2000.
- [2] Android Developers, "Room Persistence Library," <https://developer.android.com/training/data-storage/room>
- [3] K. Oflazer, "Two-level description of Turkish morphology," *Literary and Linguistic Computing*, vol. 9, no. 2, pp. 137-148, 1994.
- [4] J. Kleinberg and É. Tardos, *Algorithm Design*, Pearson, 2006.
- [5] R. Meier, *Professional Android Application Development*, Wrox, 2012.